

TPI Page Templates

Version: 1.0
Last Updated: 2025-01-XX

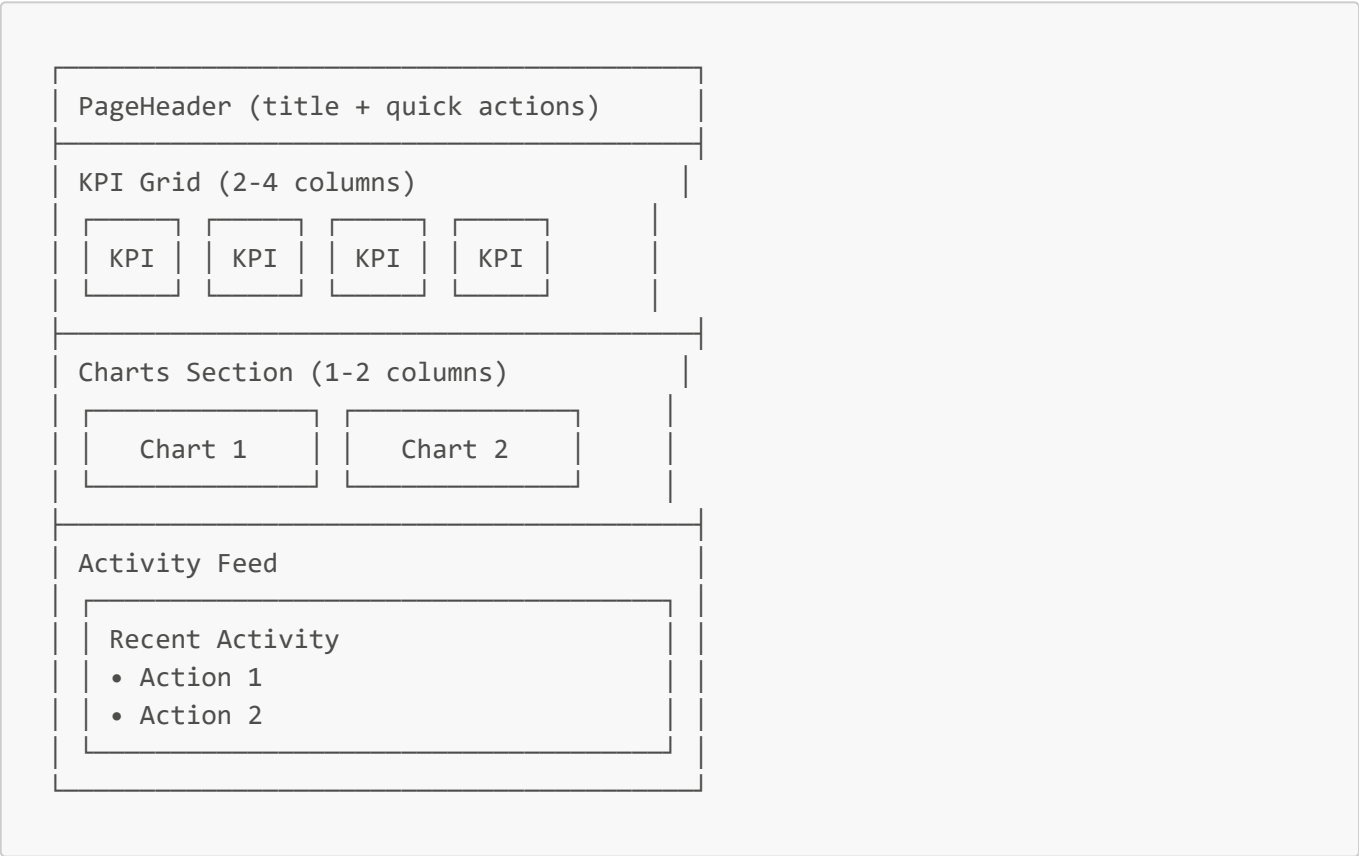
Template Overview

Six standardized page templates govern all TPI portal pages. Each template defines layout structure, component usage, and interaction patterns.

1. Dashboard Template

Purpose: Landing page for portals. Displays KPIs, charts, and activity feed.

Layout Structure:



Components Used:

- **PortalLayout** (wrapper)
- **PageHeader** (title, description, actions)
- **KpiCard** (4-6 cards in grid)
- **ActivityFeed** (recent activity)
- Charts (custom, framework-specific)

Interaction Flow:

1. User lands on dashboard
2. KPIs load first (critical data)
3. Charts load second (secondary data)
4. Activity feed loads last (tertiary data)
5. Click KPI → Navigate to detail page
6. Click activity → Navigate to related entity

Example: Technology Executive Dashboard (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title="CTO Command Center"
    description="Overview of platform health and operations"
    actions={
      <>
        <Button variant="outline">Export Report</Button>
        <Button>New Incident</Button>
      </>
    }
  />

  <div className="grid grid-cols-4 gap-4 mb-6">
    <KpiCard
      label="System Uptime"
      value={99.97}
      format="percentage"
      trend={{ value: 0.02, direction: "up", period: "vs last month" }}
    />
    <KpiCard
      label="Active Incidents"
      value={3}
      status="warning"
    />
    <KpiCard
      label="Avg Response Time"
      value="2.3s"
      format="duration"
    />
    <KpiCard
      label="Deployments This Week"
      value={12}
    />
  </div>

  <div className="grid grid-cols-2 gap-4 mb-6">
    <Card>
      <CardHeader>
        <CardTitle>Incident Trends</CardTitle>
      </CardHeader>
      <CardContent>
        <LineChart data={incidentTrends} />
      </CardContent>
    </Card>
  </div>
```

```

    </Card>
    <Card>
      <CardHeader>
        <CardTitle>Service Health</CardTitle>
      </CardHeader>
      <CardContent>
        <ServiceHealthChart data={serviceHealth} />
      </CardContent>
    </Card>
  </div>

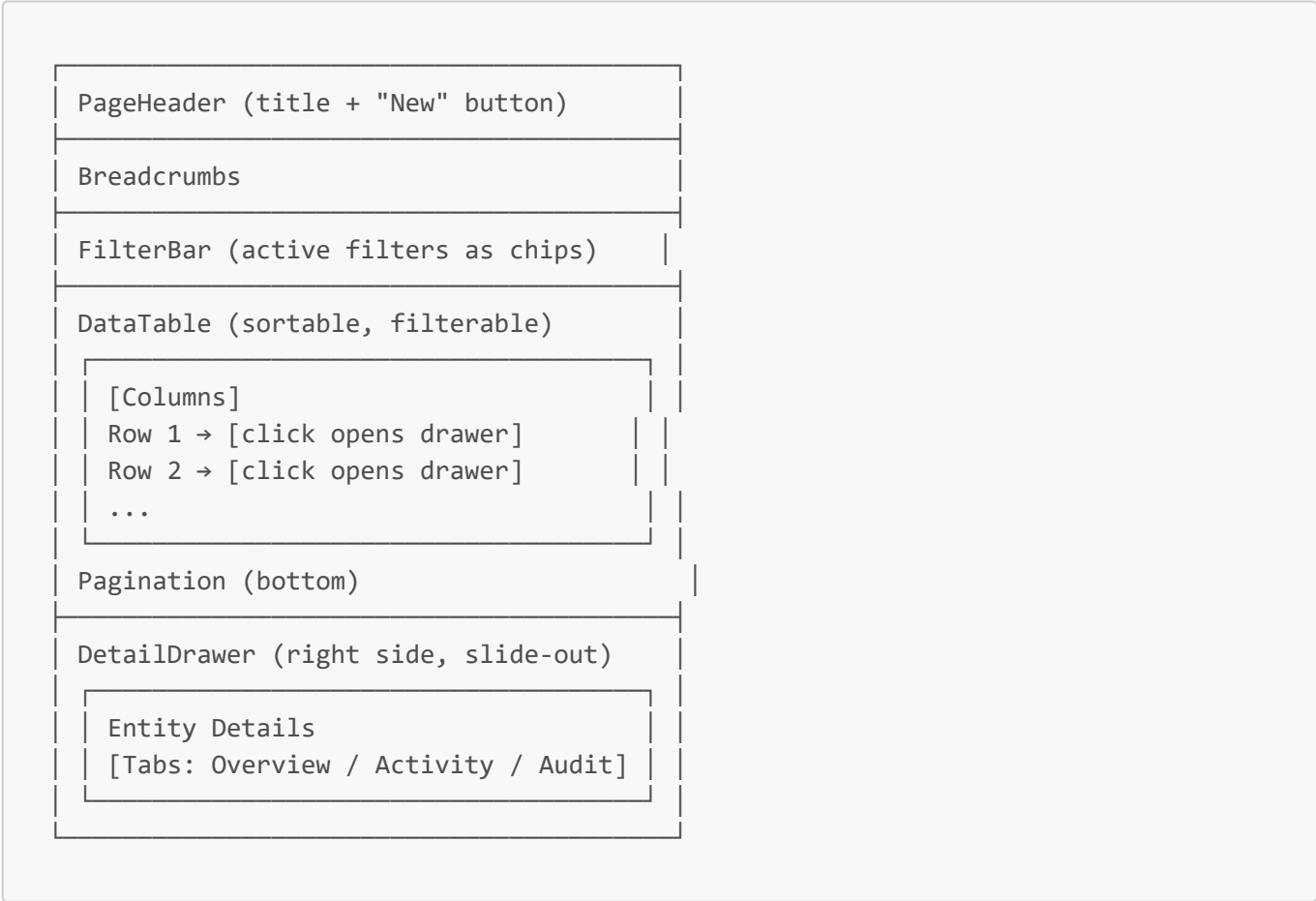
  <Card>
    <CardHeader>
      <CardTitle>Recent Activity</CardTitle>
    </CardHeader>
    <CardContent>
      <ActivityFeed activities={recentActivities} realTime />
    </CardContent>
  </Card>
</PortalLayout>

```

2. Work Management Template

Purpose: List view with table, filters, and detail drawer. Used for managing entities (incidents, tasks, etc.).

Layout Structure:



Components Used:

- `PortalLayout`
- `PageHeader`
- `Breadcrumbs`
- `FilterBar`
- `DataTable`
- `DetailDrawer`
- `Pagination` (part of `DataTable`)

Interaction Flow:

1. User views table with default filters
2. User applies filters → Table updates
3. User clicks row → Drawer opens with details
4. User edits in drawer → Save updates table
5. User closes drawer → Returns to table view
6. User clicks "New" → Full-page create form

Example: Incident Management (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title="Incident Management"
    description="Track and resolve platform incidents"
    actions={<Button>New Incident</Button>}
  />
  <Breadcrumbs items={[
    { label: "Dashboard", path: "/cto" },
    { label: "Incidents" }
  ]} />

  <FilterBar
    filters={incidentFilters}
    activeFilters={activeFilters}
    onFilterChange={setFilters}
    savedViews={savedViews}
  />

  <DataTable
    data={filteredIncidents}
    columns={incidentColumns}
    pagination={{
      page: currentPage,
      pageSize: 25,
      total: totalIncidents,
      onPageChange: setPage
    }}
    sorting={{
      column: "created_at",
      direction: "desc",
```

```

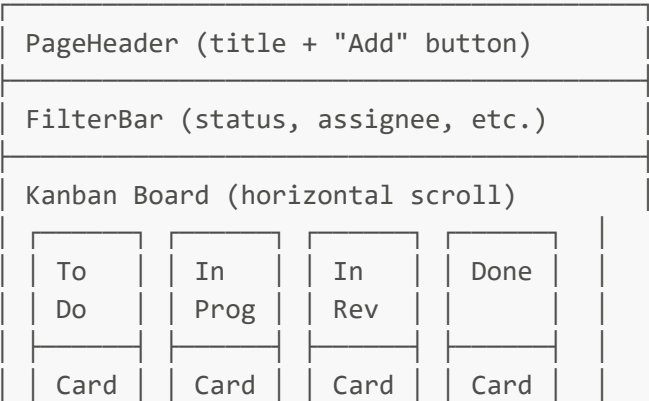
    onSort: handleSort
  }}
  onClick={row => setSelectedIncident(row.id)}
  exportable
  onExport={handleExport}
/>

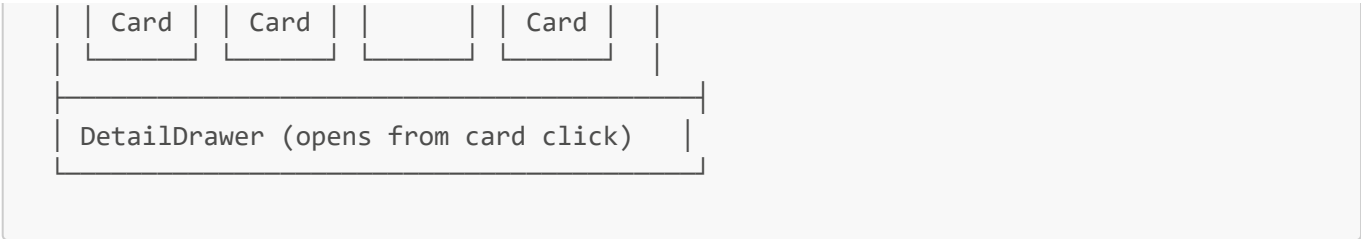
<Drawer
  open={!selectedIncident}
  onClose={() => setSelectedIncident(null)}
  title="Incident Details"
  entityId={selectedIncident}
  width={700}
>
  <Tabs defaultValue="overview">
    <Tab>Overview</Tab>
    <Tab>Activity</Tab>
    <Tab>Audit Trail</Tab>
  </Tabs>
  <TabContent value="overview">
    <IncidentOverview id={selectedIncident} />
  </TabContent>
  <TabContent value="activity">
    <ActivityFeed activities={incidentActivities} />
  </TabContent>
  <TabContent value="audit">
    <AuditTrail entityId={selectedIncident} entityType="incident" />
  </TabContent>
</Drawer>
</PortalLayout>
```

3. Kanban Template

Purpose: Board view for workflow management (tasks, releases, sprints).

Layout Structure:





Components Used:

- PortalLayout
- PageHeader
- FilterBar
- KanbanBoard (custom component)
- KanbanCard (custom component)
- DetailDrawer

Interaction Flow:

1. User views board with columns
2. User filters → Cards update
3. User drags card → Updates status (API call)
4. User clicks card → Drawer opens
5. User edits in drawer → Card updates on board
6. User clicks "Add" → Quick add form (inline or drawer)

Example: Sprint Management (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title="Sprint Management"
    description="Track engineering sprints and tasks"
    actions={<Button>New Sprint</Button>}
  />

  <FilterBar
    filters={sprintFilters}
    activeFilters={activeFilters}
    onFilterChange={setFilters}
  />

  <KanbanBoard
    columns={[
      { id: "backlog", title: "Backlog", status: "backlog" },
      { id: "todo", title: "To Do", status: "todo" },
      { id: "in-progress", title: "In Progress", status: "in-progress" },
      { id: "review", title: "In Review", status: "review" },
      { id: "done", title: "Done", status: "done" }
    ]}
    cards={sprintTasks}
    onCardMove={handleCardMove}
    onCardClick={(card) => setSelectedTask(card.id)}
  />
</PortalLayout>
```

```
    />

    <DrawerToggle
      open={!selectedTask}
      onClose={() => setSelectedTask(null)}
      title="Task Details"
      entityId={selectedTask}
    >
      <TaskDetails id={selectedTask} />
    </DrawerToggle>
  </PortalLayout>
```

4. Detail Template

Purpose: Full-page detail view with tabs (Overview, Activity, Audit).

Layout Structure:



Components Used:

- `PortalLayout`
- `PageHeader` (with back button)
- `Breadcrumbs`
- `Tabs` (framework component)
- `ActivityFeed` (Activity tab)
- `AuditTrail` (Audit tab)
- Custom detail components (Overview tab)

Interaction Flow:

1. User navigates to detail page

2. Default tab: Overview (entity details)
3. User clicks Activity tab → Activity feed loads
4. User clicks Audit tab → Audit trail loads
5. User clicks "Edit" → Form mode (inline or modal)
6. User clicks "Back" → Returns to list page

Example: Incident Detail (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title={`Incident ${incident.number}`}
    description={incident.title}
    backButton
    onBack={() => navigate("/cto/incidents")}
    actions={
      <>
        <Button variant="outline">Export</Button>
        <Button variant="outline">Edit</Button>
        <Button variant="destructive">Resolve</Button>
      </>
    }
  />
  <Breadcrumbs items={[
    { label: "Dashboard", path: "/cto" },
    { label: "Incidents", path: "/cto/incidents" },
    { label: `INC-${incident.number}` }
  ]} />

  <Tabs defaultValue="overview" className="mt-4">
    <TabsList>
      <TabsTrigger value="overview">Overview</TabsTrigger>
      <TabsTrigger value="activity">Activity</TabsTrigger>
      <TabsTrigger value="audit">Audit Trail</TabsTrigger>
    </TabsList>

    <TabsContent value="overview" className="mt-4">
      <div className="grid grid-cols-2 gap-4">
        <Card>
          <CardHeader>
            <CardTitle>Details</CardTitle>
          </CardHeader>
          <CardContent>
            <dl className="space-y-2">
              <dt className="text-sm font-medium">Status</dt>
              <dd><StatusBadge status={incident.status} label={incident.status} />
            </dd>

              <dt className="text-sm font-medium">Severity</dt>
              <dd><StatusBadge status={incident.severity} label=
{incident.severity} /></dd>

              <dt className="text-sm font-medium">Created</dt>
              <dd>{formatDate(incident.created_at)}</dd>
              <dt className="text-sm font-medium">Last Updated</dt>
```



```

        <dd>{formatDate(incident.updated_at)} by {incident.updated_by}</dd>
      </dl>
    </CardContent>
  </Card>
  <Card>
    <CardHeader>
      <CardTitle>Description</CardTitle>
    </CardHeader>
    <CardContent>
      <p>{incident.description}</p>
    </CardContent>
  </Card>
</div>
</TabsContent>

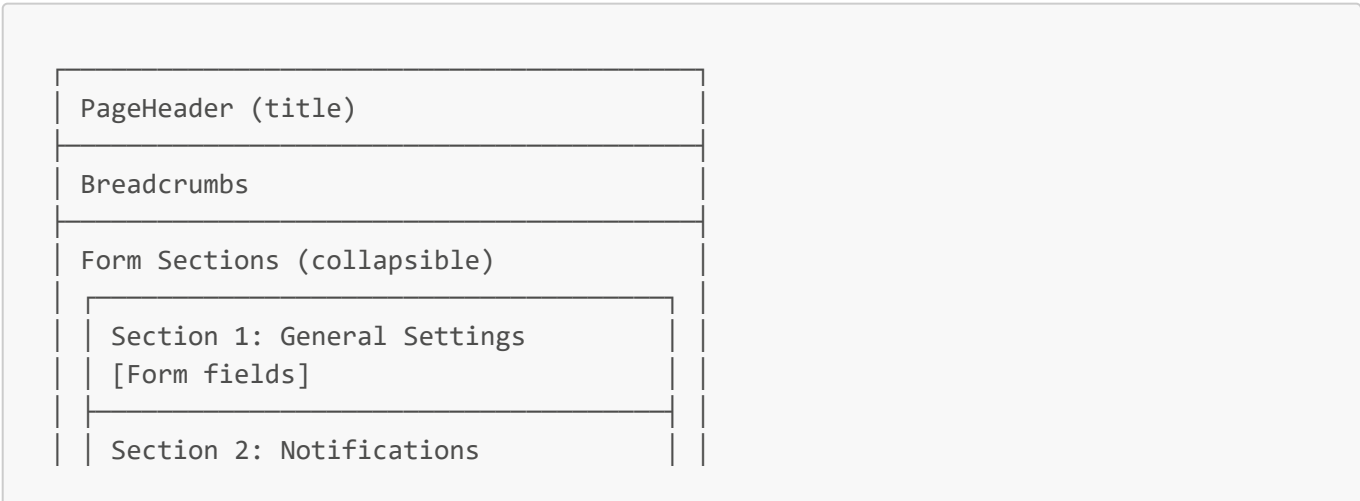
<TabsContent value="activity" className="mt-4">
  <ActivityFeed
    activities={incidentActivities}
    realTime
  />
</TabsContent>

<TabsContent value="audit" className="mt-4">
  <AuditTrail
    entityId={incident.id}
    entityType="incident"
    format="timeline"
    exportable
  />
</TabsContent>
</Tabs>
</PortalLayout>
```

5. Admin Settings Template

Purpose: Configuration pages with form sections and save bar.

Layout Structure:



[Form fields]

Section 3: Security

[Form fields]

Save Bar (sticky bottom)

[Cancel] [Save Changes]

Components Used:

- PortalLayout
- PageHeader
- Breadcrumbs
- FormSection
- FieldHelpText
- Form inputs (framework-specific)
- SaveBar (sticky footer)

Interaction Flow:

1. User views settings page
2. User edits fields → "Unsaved changes" indicator
3. User scrolls → Save bar sticks to bottom
4. User clicks "Save" → Validation → API call → Success toast
5. User clicks "Cancel" → Confirmation if unsaved changes

Example: Portal Settings (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title="Portal Settings"
    description="Configure portal behavior and preferences"
  />
  <Breadcrumbs items={[
    { label: "Dashboard", path: "/cto" },
    { label: "Settings" }
  ]} />

  <form onSubmit={handleSubmit} className="space-y-6">
    <FormSection
      title="General Settings"
      description="Basic portal configuration"
    >
      <Input
        label="Portal Name"
        value={settings.portalName}
        onChange={(e) => setSettings({ ...settings, portalName: e.target.value })}
      />
```

```

    <FieldHelpText text="Display name shown in navigation" />

    <Select
      label="Default View"
      value={settings.defaultView}
      onChange={(value) => setSettings({ ...settings, defaultView: value })}
    >
      <SelectItem value="dashboard">Dashboard</SelectItem>
      <SelectItem value="list">List View</SelectItem>
    </Select>
  </FormSection>

  <FormSection
    title="Notifications"
    description="Configure notification preferences"
    collapsible
  >
    <Switch
      label="Email Notifications"
      checked={settings.emailNotifications}
      onChange={(checked) => setSettings({ ...settings, emailNotifications:
checked })}
    />
    <Switch
      label="Push Notifications"
      checked={settings.pushNotifications}
      onChange={(checked) => setSettings({ ...settings, pushNotifications:
checked })}
    />
  </FormSection>

  <FormSection
    title="Security"
    description="Security and access settings"
  >
    <Input
      label="Session Timeout (minutes)"
      type="number"
      value={settings.sessionTimeout}
      onChange={(e) => setSettings({ ...settings, sessionTimeout:
parseInt(e.target.value) })}
    />
    <FieldHelpText text="Automatically log out users after inactivity"
variant="warning" />
  </FormSection>
</form>

<SaveBar
  hasUnsavedChanges={hasUnsavedChanges}
  onSave={handleSave}
  onCancel={handleCancel}
  loading={saving}
/>
</PortalLayout>

```

6. Incident/Runbook Template

Purpose: Timeline-based view for incidents with actions and postmortem.

Layout Structure:



Components Used:

- PortalLayout
- PageHeader
- Breadcrumbs

- **StatusBadge**
- **Timeline** (custom component)
- **ActivityFeed** (timeline events)
- **Card** (status, actions, postmortem)

Interaction Flow:

1. User views incident detail
2. User sees timeline of events
3. User takes action → Action added to timeline
4. User resolves incident → Postmortem section appears
5. User fills postmortem → Saves for future reference

Example: Incident Response (CTO Portal)

```
<PortalLayout portalName="Technology Executive Dashboard">
  <PageHeader
    title={`Incident ${incident.number}`}
    description={incident.title}
    badge={<StatusBadge status={incident.severity} label={incident.severity} />}
    actions={
      <>
        <Button variant="outline">Acknowledge</Button>
        <Button variant="outline">Escalate</Button>
        <Button>Resolve</Button>
      </>
    }
  />
  <Breadcrumbs items={[
    { label: "Dashboard", path: "/cto" },
    { label: "Incidents", path: "/cto/incidents" },
    { label: `INC-${incident.number}` }
  ]} />

  <div className="grid grid-cols-3 gap-4 mt-4">
    <div className="col-span-2 space-y-4">
      <Card>
        <CardHeader>
          <CardTitle>Incident Status</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="grid grid-cols-3 gap-4">
            <div>
              <dt className="text-sm font-medium">Status</dt>
              <dd><StatusBadge status={incident.status} label={incident.status} />
            </dd>
          </div>
          <div>
            <dt className="text-sm font-medium">Severity</dt>
            <dd><StatusBadge status={incident.severity} label=
{incident.severity} /></dd>
          </div>
        </CardContent>
      </Card>
    </div>
  </div>
```

```

        <div>
          <dt className="text-sm font-medium">Assigned To</dt>
          <dd>{incident.assigned_to?.name || "Unassigned"}</dd>
        </div>
      </div>
    </CardContent>
  </Card>

  <Card>
    <CardHeader>
      <CardTitle>Timeline</CardTitle>
    </CardHeader>
    <CardContent>
      <Timeline events={incidentTimeline} />
    </CardContent>
  </Card>

  {incident.status === "resolved" && (
    <Card>
      <CardHeader>
        <CardTitle>Postmortem</CardTitle>
      </CardHeader>
      <CardContent>
        <FormSection title="Root Cause">
          <Textarea value={incident.root_cause} readOnly />
        </FormSection>
        <FormSection title="Impact Assessment">
          <Textarea value={incident.impact} readOnly />
        </FormSection>
        <FormSection title="Prevention Measures">
          <Textarea value={incident.prevention} readOnly />
        </FormSection>
      </CardContent>
    </Card>
  )}
</div>

<div className="space-y-4">
  <Card>
    <CardHeader>
      <CardTitle>Quick Actions</CardTitle>
    </CardHeader>
    <CardContent className="space-y-2">
      <Button className="w-full" variant="outline">Acknowledge</Button>
      <Button className="w-full" variant="outline">Escalate</Button>
      <Button className="w-full" variant="outline">Assign</Button>
      <Button className="w-full" variant="destructive">Resolve</Button>
    </CardContent>
  </Card>

  <Card>
    <CardHeader>
      <CardTitle>Related Incidents</CardTitle>
    </CardHeader>

```

```
        <CardContent>
          <ul className="space-y-2">
            {relatedIncidents.map(inc => (
              <li key={inc.id}>
                <Link to={`/cto/incidents/${inc.id}`}>
                  {inc.number} - {inc.title}
                </Link>
              </li>
            ))}
          </ul>
        </CardContent>
      </Card>
    </div>
  </div>
</PortalLayout>
```

Template Selection Guide

Use Dashboard Template when:

- Portal landing page
- Overview of multiple metrics
- Activity monitoring

Use Work Management Template when:

- Managing list of entities
- Need filtering/sorting
- Detail view is secondary

Use Kanban Template when:

- Workflow-based management
- Status progression tracking
- Visual workflow representation

Use Detail Template when:

- Primary content is entity details
- Multiple related views (tabs)
- Deep-dive information

Use Admin Settings Template when:

- Configuration pages
- Form-heavy content
- Save/cancel workflow

Use Incident/Runbook Template when:

- Time-based events

- Action-oriented workflows
 - Postmortem documentation
-

Template Customization

Templates are guidelines, not strict rules. Customize when:

- Portal has unique requirements
- User research indicates different patterns
- Performance requires optimization

Always maintain:

- Consistent spacing
 - Standard component usage
 - Accessibility compliance
 - Navigation patterns
-

Next: Review [Pattern Guides](#) for detailed interaction patterns.